BANQUETBAZAR

SANDIP PATEL
(LCID: LC00017002051)

FINAL YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT FOR THE DEGREE OF BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

FACULTY OF COMPUTER SCIENCE IN MULTIMEDIA
LINCOLN UNIVERSITY COLLEGE
MALAYSIA

20th April, 2026

Declaration

I hereby declare that this report is based on my own independent work, except for Quotation and summaries which have been dully acknowledged. I also declare that no part of this work has been submitted for any degree to this or any other university.

ACKNOWLEDGEMENT

First of all, I thank Almighty for inspiring me with strength and energy to achieve this modest study. I am deeply grateful to all those who have encouraged and helped me, discussed ideas and insights, which have contributed in various ways to complete my study.

My deepest gratitude and appreciation go to my supervisor Er. Rohit Pandey for his patience and very helpful comments, her bright ideas, guidance and for being generous with her knowledge and experience in supervising

I would like to express my sincere gratitude to

ABSTRACT

BanquetBazar is a web-based AI-powered banquet management system developed to modernize the traditional process of booking banquet halls for events such as weddings, birthday parties, meetings, and other social gatherings. The current manual booking system is not only time-consuming but are also scattered and does not assist users in choosing the best option. This project aims to provide a centralized digital platform where users can browse available halls, check date availability and request bookings easily. The system integrates artificial intelligence techniques to improve decision-making by recommending suitable banquet halls based on budget, guest count, and event type. The system is developed using the Incremental Development Model, with React.js for the frontend and Django with Django REST Framework for the backend, supported by a PostgreSQL database. Testing results show that the platform simplifies the booking process, improves transparency, and provides reliable recommendations. Overall, the integration of web technologies and artificial intelligence enhances efficiency, user experience, and management effectiveness in banquet hall booking.

Table of Contents

CHAPTER 1: INTRODUCTION 1
1.1 Background of the Study 1
1.2 Problem Statement 1
1.3 Objectives 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Scope of the Study 2
1.4.1 Included in the Scope: 2
1.4.2 Excluded from the Scope: 2
1.5 Significance of the Study 2
1.6 Limitations 4
1.7 Organization of the Report 4
CHAPTER 2: LITERATURE REVIEW 5
2.1 Introduction 5
2.2 Review of Related Works 5
2.3 Theoretical Framework 6
2.4 Research Gap 6
2.5 Summary 6
CHAPTER 3: METHODOLOGY 7
3.1 Research/Development Method 7
3.2 System Requirement Specification (SRS) 7
3.2.1 Functional Requirements 7
3.2.2 Non-functional Requirements 8
3.3 Tools & Technologies Used 9
3.4 Feasibility Study 10
3.5 System Architecture 11
CHAPTER 4: SYSTEM DESIGN 13
4.1 Use Case Diagram & Description 13
4.1.1 Customer Use Cases 14
4.1.2 Admin Use Cases 14
4.2 DFD (Level 0, 1, 2) 15
4.3 ER Diagram 17

4.4 Class Diagram 18
4.5 Sequence/Activity Diagrams 19
4.6 UI/UX Design 20
CHAPTER 5: SYSTEM IMPLEMENTATION 21
5.1 Module Description 21
5.2 Frontend Implementation 23
5.3 Backend Implementation 24
5.4 Database Implementation 26
5.5 Algorithm Used 28
5.6 Screenshots of System Features 30
CHAPTER 6: TESTING AND RESULT ANALYSIS 31
6.1 Test Plan 31
6.2 Test Cases (Table Format) 33
6.3 Unit Testing 36
6.4 Integration Testing 36
6.5 System Testing 38
6.6 Performance Evaluation 40
CHAPTER 7: CONCLUSION AND RECOMMENDATION 41
7.1 Summary 41
7.2 Conclusion 42
7.3 Limitations 43
7.4 Future Enhancements 43

List of Figures

Figure 1: System Architecture 12
Figure 2: Use Case Diagram 13
Figure 3: DFD-Level 0 15
Figure 4: DFD- Level 1 15
Figure 5: DFD- Level 2 16
Figure 6: ER- Diagram 17
Figure 7: Class Diagram..18
Figure 8: Sequential Diagram 19
Figure 9: UI/UX Design..20

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The process of booking banquet halls for events such as weddings, birthday parties, meetings, and social gatherings is still largely carried out through traditional methods, including phone calls, physical visits, and social media messages. These methods are often time-consuming, scattered, and require customers to manually compare hall availability, pricing, and services. As a result, planning events becomes complicated, inefficient, and prone to miscommunication or delays in booking.

With the advancement of web technologies and artificial intelligence, it is possible to develop intelligent systems that simplify the banquet booking process. By integrating AI-based recommendations and a web-based platform can help users choose suitable halls, plan their events efficiently, and save time. BanquetBazar is designed as a centralized, AI-powered banquet management system that addresses these issues by providing a transparent, user-friendly, and effective solution for both customers and hall administrators.

1.2 Problem Statement

Booking banquet halls for events through traditional methods is often inefficient, time-consuming, and confusing, as customers must contact multiple halls via phone calls, visit them physically, or rely on scattered social media pages to check availability, pricing, and services. This makes it difficult to compare options, calculate costs accurately, and make informed decisions. Additionally, there is no intelligent system that can recommend suitable halls or services based on the user's budget, guest count, and event type. Therefore, there is a need for a web-based AI-powered banquet management system that centralizes all information, streamlines the booking process, and provides smart recommendations and automatic cost estimation to assist users in planning their events more efficiently.

1.3 Objectives

1.3.1 General Objective

To develop a web-based AI-powered banquet management system that makes it easier for users to find, compare, and book banquet halls while providing smart recommendations.

1.3.2 Specific Objectives

a) To allow customers to browse banquet halls and view details such as price, capacity, and services.

b) To enable customers to check hall availability and request bookings online.

1.4 Scope of the Study

The scope of this study defines the boundaries and focus of the BanquetBazar system. The project is designed to provide a centralized platform for booking banquet halls with AI assistance. The system will include the following features:

1.4.1 Included in the Scope:

a) Customer registration and secure login for accessing the platform.

b) Browsing and viewing details of banquet halls including price, capacity, photos, and services.

c) Checking date availability for each hall and requesting bookings online.

d) Selecting additional event services such as decoration, catering, photography, and DJ.

e) AI-based hall recommendations and automatic cost estimation to help users plan efficiently.

f) Customer dashboard to track booking status and view past bookings.

g) Admin panel for banquet owners to manage halls, services, and booking approvals.

1.4.2 Excluded from the Scope:

a) Online payment or integration with payment gateways.

b) Real-time chat or direct messaging between customers and hall owners.

c) Mobile application version.

d) Multi-branch management for banquet owners.

e) Notifications via SMS or WhatsApp.

1.5 Significance of the Study

The BanquetBazar system provides practical benefits for customers, banquet owners, society, and academic learning. By integrating AI recommendations and a complete admin panel, the system enhances both user experience and management efficiency.

For Customers:

a) Offers a centralized platform to browse banquet halls, services, and prices.
b) Saves time because one can see the real time availability and then plan an event.
c) Reduces confusion by providing AI-based recommendations for the best hall and services according to budget and preferences.
d) Provides a transparent and easy booking process, improving user confidence.

For Event Service Providers (Indirect Benefit):

a) Increases visibility of halls and services through an organized, digital platform.
b) Reduces unnecessary inquiries because customers can view all details online before booking.

For Society

a) Supports digital transformation in the event planning sector.
b) Encourages organized event management, reducing manual communication and misunderstandings.
c) Promotes convenience and smarter decision-making for families planning events.

Academic Significance

a) Shows the practical use of the full-stack web application with the help of the latest technologies.
b) Shows practical applications of AI models which include recommendation systems and cost prediction.
c) Gives students practical skills on how to construct scalable systems that have frontend, back end, and database integration.
d) Provides a base of further improvements, such as automation, analytics, and the creation of mobile applications.

1.6 Limitations

While BanquetBazar provides an efficient and modern solution for banquet hall booking, the system has certain technical and functional limitations in its current phase:

a) No Mobile App Version: The system is web-based only; users cannot access it through a dedicated mobile application.

b) Payment Gateway Integration Excluded: Online payments and secure transactions are not implemented.

c) Real-Time Communication Not Supported: There is no live chat or direct messaging between customers and hall owners.

d) Limited Admin Features: Multi-branch management and advanced analytics for admins are not included.

e) AI Model Limitations: Recommendations and cost estimations rely on available historical data and basic models; accuracy may vary with incomplete or inconsistent data.

f) Scalability Constraints: The system is designed for a limited number of halls and bookings; very high traffic may require performance optimization.

1.7 Organization of the Report

This report is organized into four chapters.

Chapter 1 covers the introduction, objectives, scope, and limitations.

Chapter 2 This section includes a fully-referenced review and discussions of previous studies which are relevant to the project

Chapter 3 describes the development methodology, system requirements, tools, and architecture.

Chapter 4 presents the full system design, including use case diagrams, data flow diagrams, an entity-relationship diagram, a class diagram, sequence and activity diagrams, and the UI/UX design.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

A literature review provides an overview of existing studies, systems, and technologies relevant to a project. It helps identify gaps, challenges, and best practices that guide the development of a new solution. In banquet hall management, previous systems often focus on manual booking processes or internal hotel operations, lacking intelligent features such as AI-based recommendations or automatic cost estimation. This chapter reviews existing literature on digital banquet management, online booking systems, and AI-assisted recommendation systems to form a foundation for developing BanquetBazar, a web-based AI-powered platform for efficient and user-friendly banquet booking.

2.2 Review of Related Works

Several studies and projects have explored digital solutions for banquet hall and event management:

1) Orazova & Permanova (2025) proposed a digital system for managing banquet halls, focusing on event planning, booking, and resource management. The system improved operational efficiency but did not include AI-based recommendations for customers.

2) Yalini (2017) developed an online booking system for Loyolas Wedding Hall, allowing users to book halls, select service packages, and generate reports. While it improved accessibility, it was built with PHP and MySQL, limiting scalability and integration with modern AI technologies.

3) Lalitha, R., Kumar, S., & Priya, M. (2022): Designed and implemented an online event hall booking and reservation system, improving accessibility and management efficiency for users and admins.

4) Islam, F. (2024): Developed AI-driven modern web applications, highlighting how AI can enhance web-based systems and improve user experience in event management.

These studies show that while digital and automated systems exist, there is still a gap in providing a centralized, AI-assisted, customer-focused banquet booking platform.

2.3 Theoretical Framework

The theoretical framework of BanquetBazar combines concepts from:

i. Web-based Information Systems : providing centralized access for users and admins.
ii. Artificial Intelligence (AI) : using Random Forest for hall recommendations and rule-based methods for cost estimation.
iii. Incremental Development Model : guiding the development process in stages to allow early testing, feedback, and integration of features.

This framework ensures that the system is both technically robust and aligned with user needs for efficient, transparent, and intelligent banquet booking.

2.4 Research Gap

Existing studies and systems for banquet management and event planning reveal several gaps:

i. Lack of AI-based recommendation systems for hall and service selection.
ii. Absence of automatic cost estimation features.
iii. Limited focus on customer-centered digital platforms; many systems prioritize internal hotel operations.
iv. Fragmented and scattered manual booking processes that are still widely used.

BanquetBazar aims to address these gaps by providing a centralized, AI-powered platform that streamlines the booking process, improves decision-making, and enhances user experience.

2.5 Summary

In summary, previous literature shows that while digital and automated systems have improved banquet hall management, there are still limitations in intelligence, customer focus, and centralized access. This study identifies the need for a web-based AI-powered platform that integrates hall browsing, AI recommendations, and automatic cost estimation, providing a more efficient and user-friendly solution for banquet hall booking.

CHAPTER 3: METHODOLOGY

3.1 Research/Development Method

The project uses an Incremental Development Model, delivering the system in functional increments to allow early testing, feedback, and risk reduction.

a) Increment 1: User authentication (customer and admin login/signup) and basic UI setup.
b) Increment 2: Admin panel for managing halls, services, and booking approvals.
c) Increment 3: Hall browsing module with hall details, images, and services.
d) Increment 4: Booking module for checking availability, selecting halls and services, and submitting booking requests.
e) Increment 5: AI recommendation module for suggesting suitable halls based on budget, guest count, and event type, along with automatic cost estimation.
f) Increment 6: Final UI enhancements, system testing, bug fixing, and deployment.

3.2 System Requirement Specification (SRS)

3.2.1 Functional Requirements

Functional requirements define the specific behavior and functions of the system that the users and admins can perform. For BanquetBazar, these include:

1) User Authentication:
   i. Customers and admins can securely register and log in to the system.
   ii. Role-based access control to differentiate customer and admin functionalities.
2) Hall Browsing:
   i. Customers can view a list of banquet halls with details such as images, pricing, capacity, and available services.
3) Booking Management:
   i. Customers can check hall availability for selected dates.
   ii. Customers can select services and request bookings.
   iii. Admins can approve, reject, or add remarks to booking requests.
4) AI Recommendation Module:
   i. The system suggests suitable halls based on user budget, guest count, and event type.

ii. Automatic cost estimation is provided for the selected hall and services.

5) Admin Panel:

i. Admins can manage halls, services, and booking records.

ii. Admins can generate reports for bookings and hall usage.

6) Customer Dashboard:

i. Customers can view past and upcoming bookings.

ii. Display AI recommendations and estimated costs.

3.2.2 Non-functional Requirements

Non-functional requirements define the quality and constraints of the system, including performance, security, and usability:

Performance:

i. The system should respond to user requests within 2:3 seconds under normal load.

ii. Must handle multiple concurrent users without crashing.

Usability:

i. User interface should be intuitive, responsive, and easy to navigate on web browsers.

ii. Minimal learning curve for both customers and admins.

Security:

i. Secure login with encrypted passwords.

ii. Role-based access to prevent unauthorized access to admin functions.

Reliability:

i. The system should be available 99% of the time.

ii. Error handling to prevent system crashes.

Scalability:

i. Capable of supporting additional halls, services, and users in future expansions.

Maintainability:

ii. Code should be modular and well-documented for easier updates and bug fixes.

3.3 Tools & Technologies Used

Tools and Technologies

1) Programming Languages
   i. JavaScript: For building the React.js frontend
   ii. Python: For Django backend and AI/ML models

2) Frontend Technologies
   i. React.js: For creating a responsive and dynamic user interface
   ii. React Router: For page navigation
   iii. Axios: For communicating with backend APIs
   iv. Tailwind CSS: For UI styling and layout

3) Backend Technologies
   i. Django (Python): For backend logic, authentication, and API development
   ii. Django REST Framework (DRF): For creating RESTful APIs consumed by the frontend

4) Database
   i. PostgreSQL: Relational database for storing users, halls, services, and bookings

5) AI / Machine Learning Tools
   i. scikit-learn: For Random Forest recommendations and Linear Regression cost estimation
   ii. NumPy & Pandas: For data preprocessing and model training

6) Development Tools
   i. VS Code: Main editor for frontend and backend development
   ii. Figma: For UI/UX design

3.4 Feasibility Study

A feasibility study evaluates whether the project is practical and achievable within the given resources, time, and technology. For BanquetBazar, the feasibility is assessed in three main areas:

1) Technical Feasibility

   The project uses proven technologies such as React.js for the frontend, Django and Django REST Framework for the backend, and PostgreSQL for the database. AI features are implemented using Python libraries such as scikit-learn, NumPy, and Pandas. The system does not require highly complex technology beyond the team's current skills. Therefore, it is technically feasible to develop and deploy the system.

2) Economic Feasibility

   The project requires minimal cost as it uses open-source software (React.js, Django, PostgreSQL, Python) and free cloud hosting options (Vercel for frontend, Heroku for backend). The main investment is time and human resources, which are manageable within the project timeline. Hence, the project is economically feasible.

3) Operational Feasibility

   The system is designed to be user-friendly and intuitive for both customers and admins. Users can easily browse halls, request bookings, and receive AI-based recommendations without extensive training. Admins can manage halls and bookings efficiently. This ensures the system can be operated effectively in real-world scenarios.

3.5 System Architecture

BanquetBazar uses a decoupled architecture, where the frontend, backend, AI module, and database operate independently but communicate through APIs. This design improves flexibility, scalability, and maintainability.

a) Frontend (React.js):

i. Handles user interfaces for customers and admins.

ii. Manages hall browsing, booking requests, service selection, dashboards, and AI recommendation display.

b) Backend (Django & Django REST Framework):

i. Provides secure APIs for frontend communication.

ii. Handles authentication, booking management, hall and service data, and business logic.

c) AI Module (Python):

i. Uses Random Forest for hall recommendations based on user budget, guest count, and event type.

ii. Provides rule-based automatic cost estimation for selected halls and services.

d) Database (PostgreSQL):

i. Stores all system data including users, halls, services, bookings, and AI training datasets.

ii. Ensures data integrity, security, and fast retrieval for both backend and AI module.

e) External Data (Optional / Future Integration):

i. Allows integration with APIs for event services, payment gateways, or notifications in future enhancements.

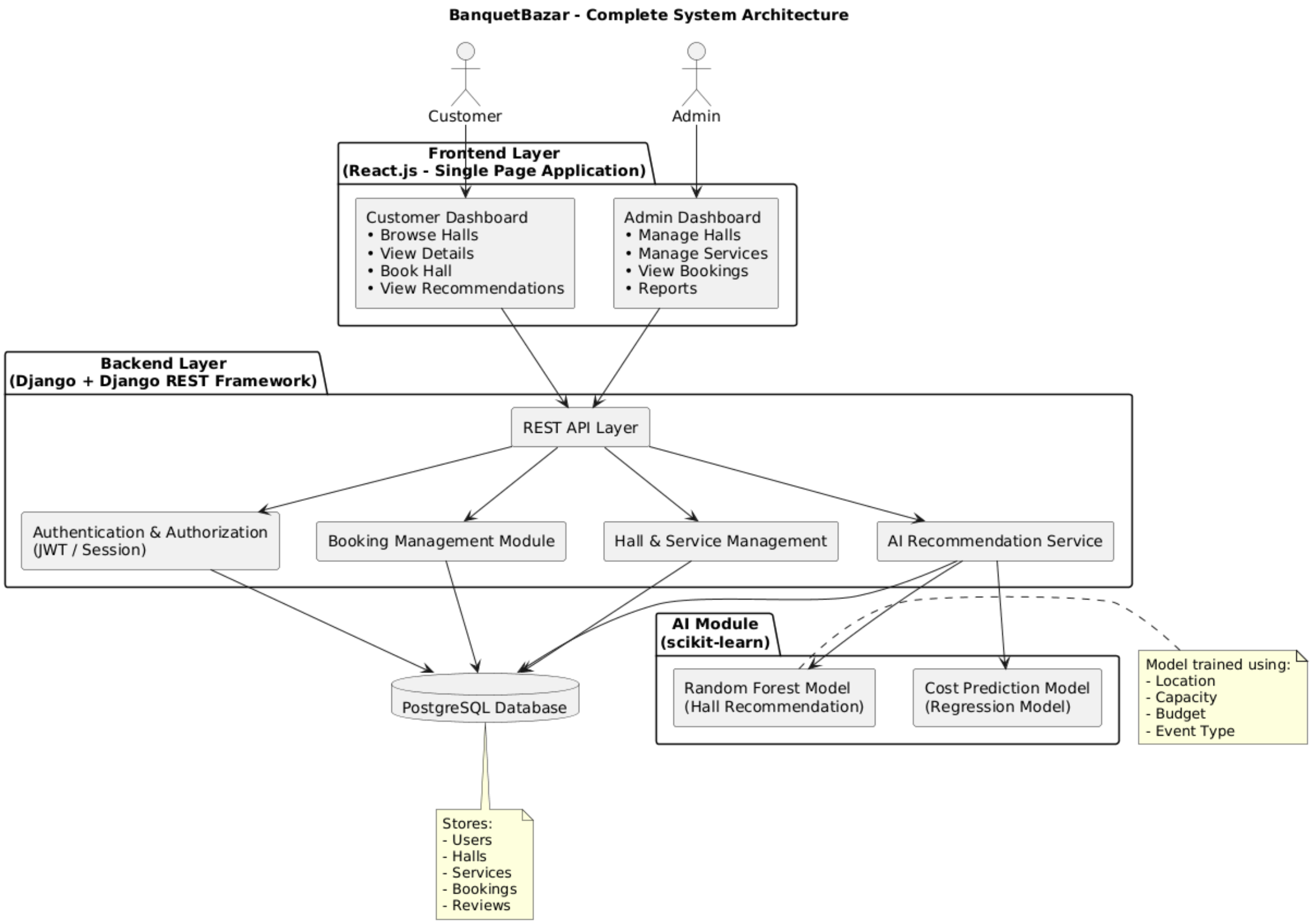

Figure 1: System Architecture

CHAPTER 4: SYSTEM DESIGN

4.1 Use Case Diagram & Description

The Use Case Diagram of BanquetBazar System shows the interaction between two actors: Customer and Admin**.**

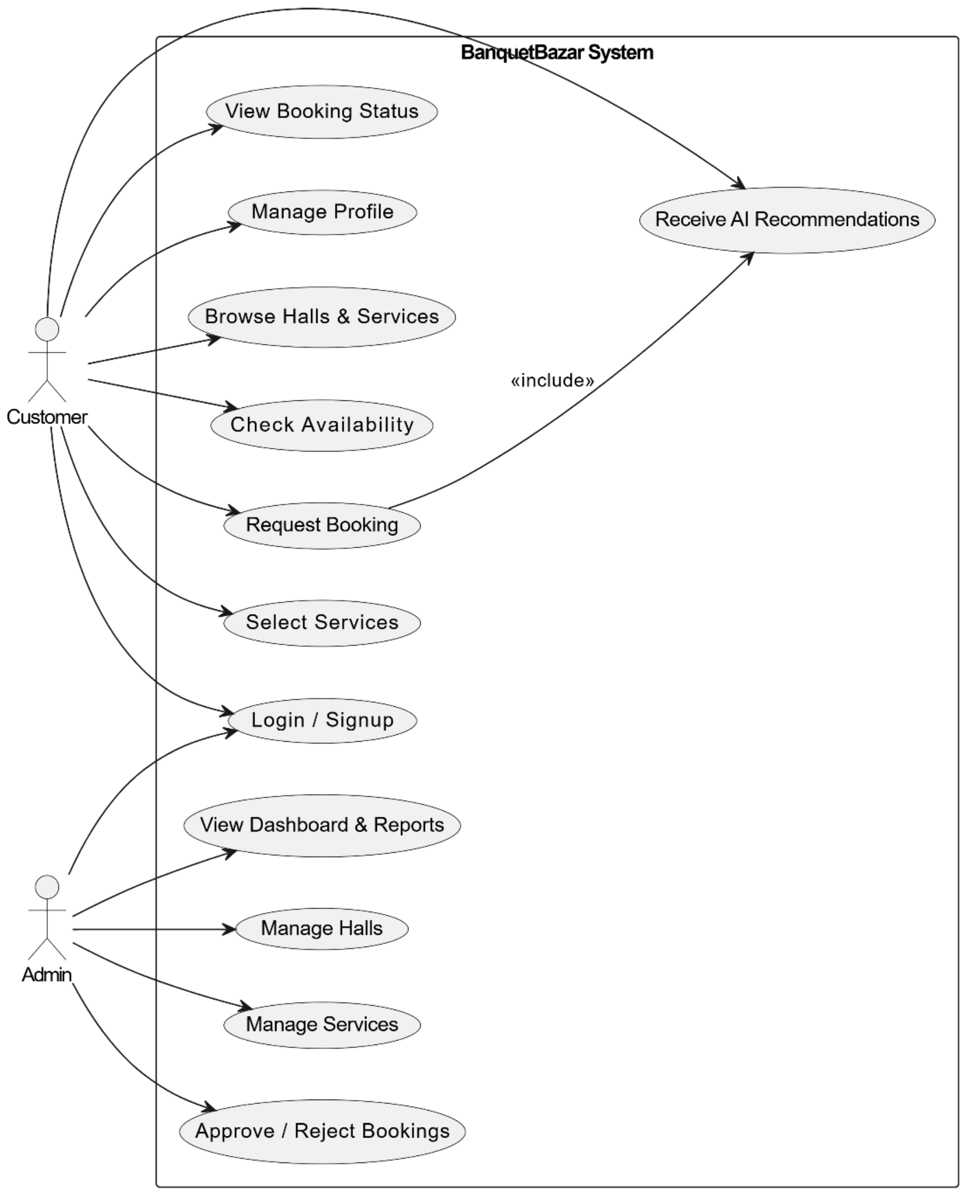

Figure 2: Use Case Diagram

Actors

1) Customer:

   User who searches halls, requests bookings, selects services, and receives AI recommendations.

2) Admin:

   Manages halls, services, bookings, and views reports.

4.1.1 Customer Use Cases

i. Login / Signup: Access the system.
ii. Manage Profile: Update personal details.
iii. Browse Halls & Services: View available halls and services.
iv. Check Availability: Check-hall availability for selected date.
v. Select Services: Choose additional services.
vi. Request Booking: Send booking request.
vii. Receive AI Recommendations: Get hall suggestions based on budget, guests, and event type.
viii. View Booking Status: Check booking approval status.

4.1.2 Admin Use Cases

i. Login: Access admin panel.
ii. Manage Halls: Add, update, or delete hall details.
iii. Manage Services: Manage service information.
iv. Approve / Reject Bookings: Process booking requests.
v. View Dashboard & Reports: Monitor system statistics.

4.2 DFD (Level 0, 1, 2)

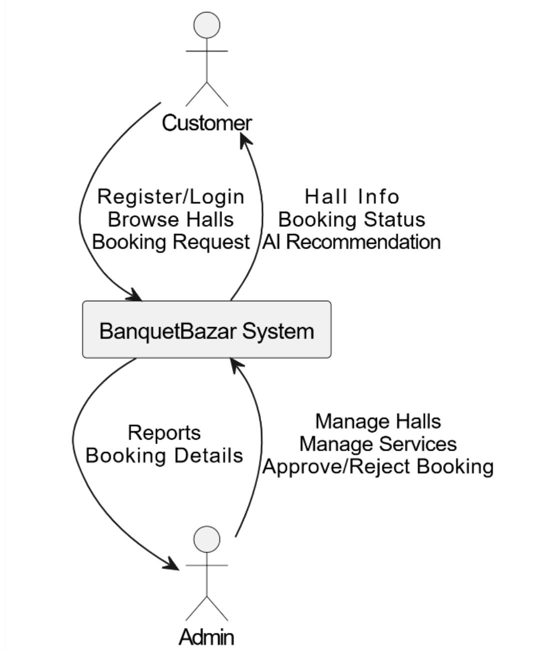

Figure 3: DFD-Level 0

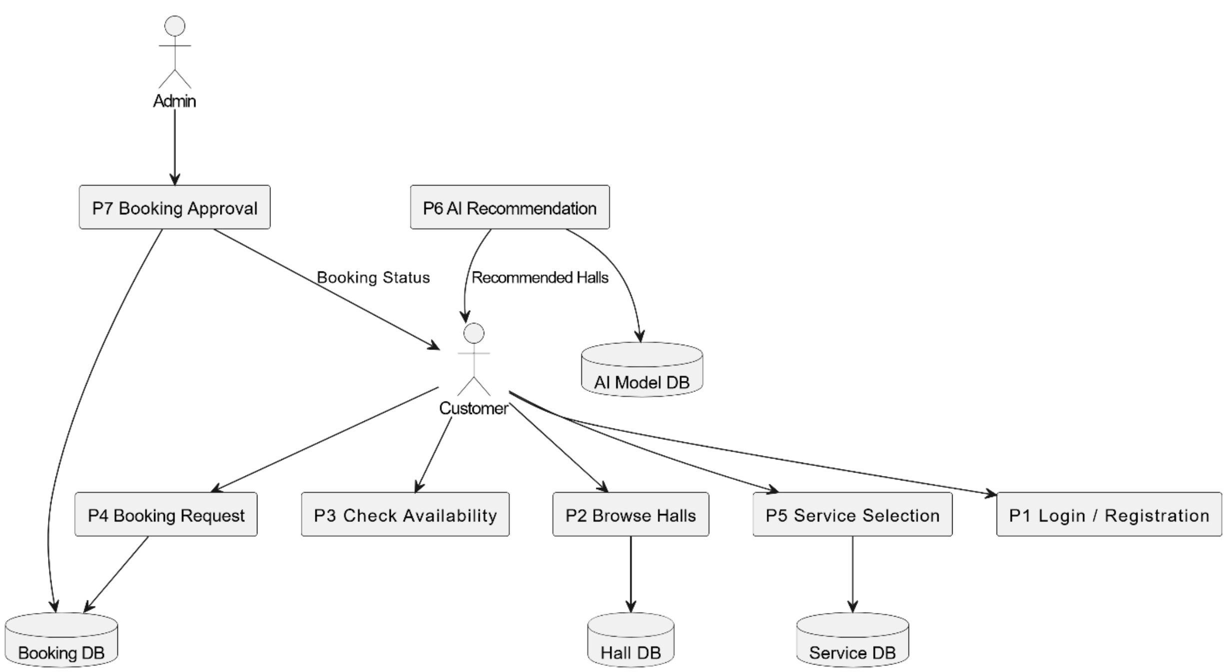

Figure 4: DFD- Level 1

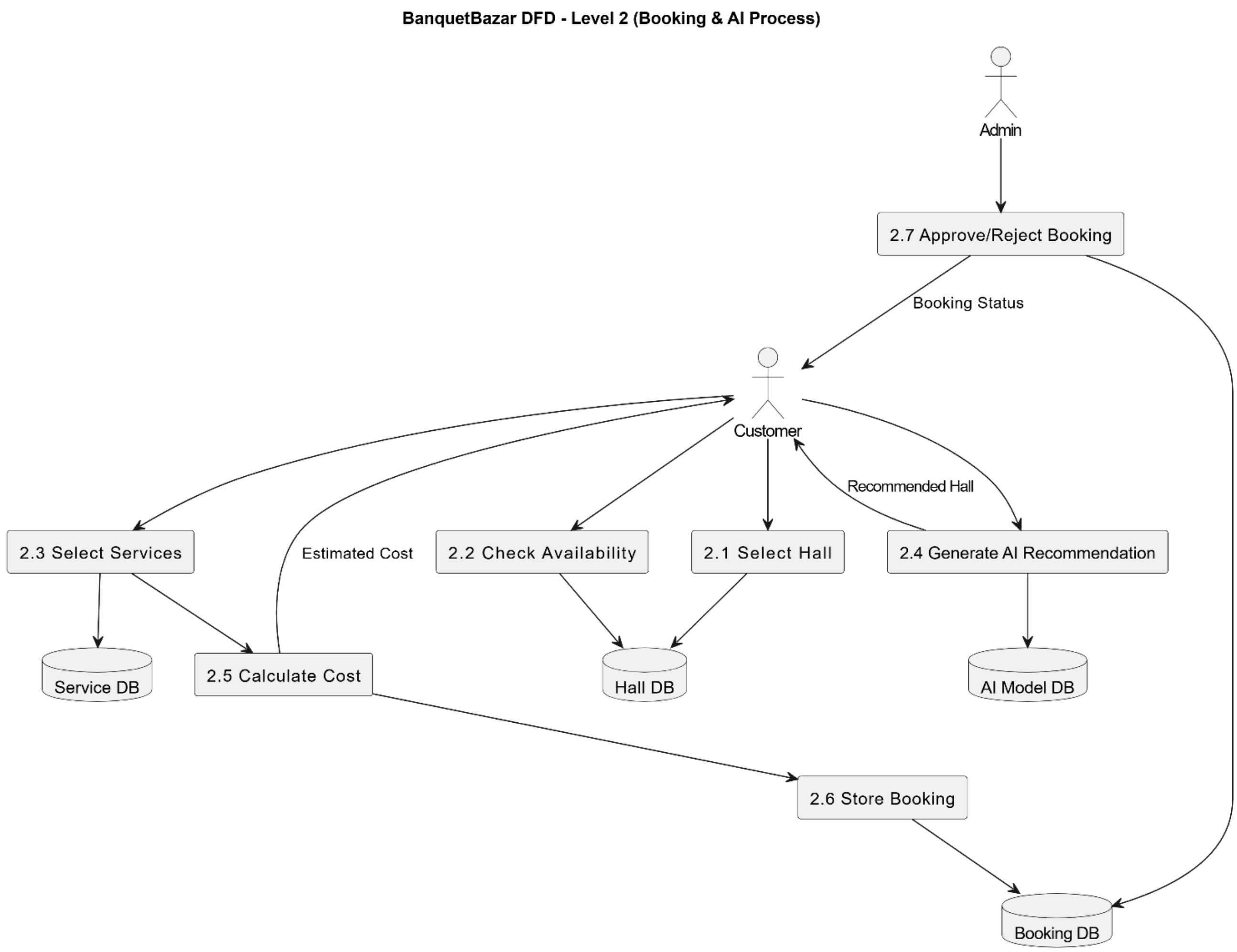

Figure 5: DFD- Level 2

4.3 ER Diagram

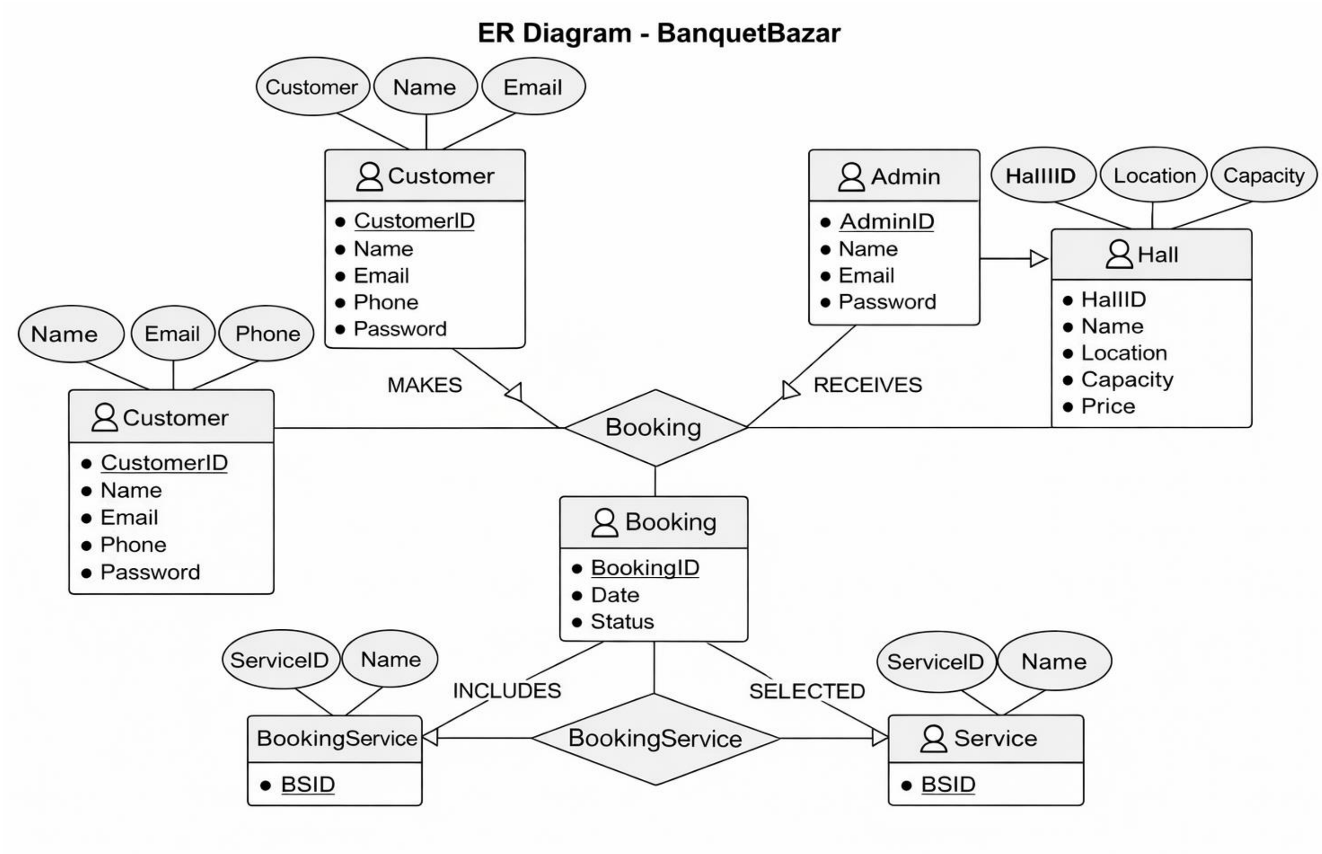

Figure 6: ER- Diagram

4.4 Class Diagram

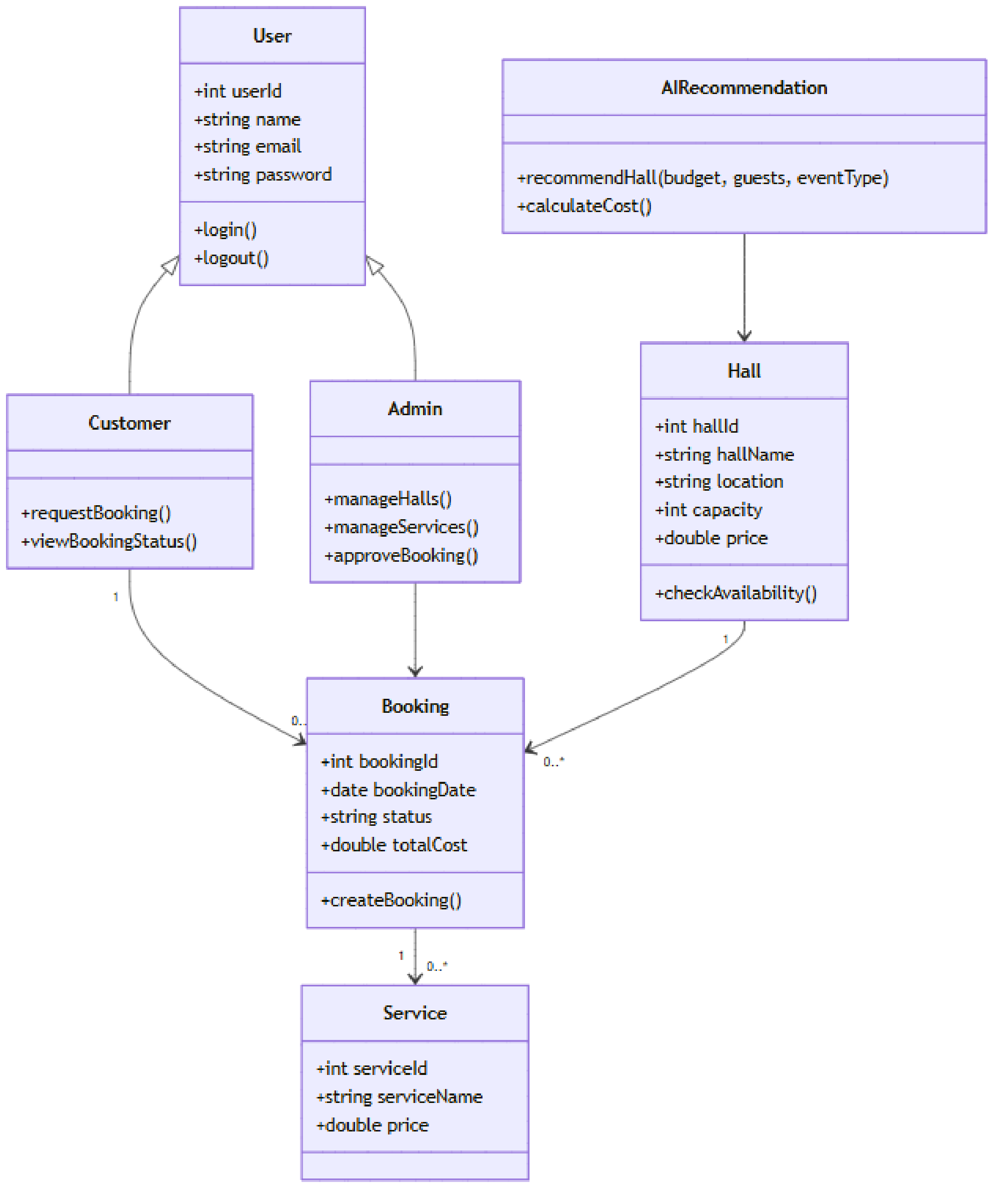

Figure 7: Class Diagram

4.5 Sequence/Activity Diagrams

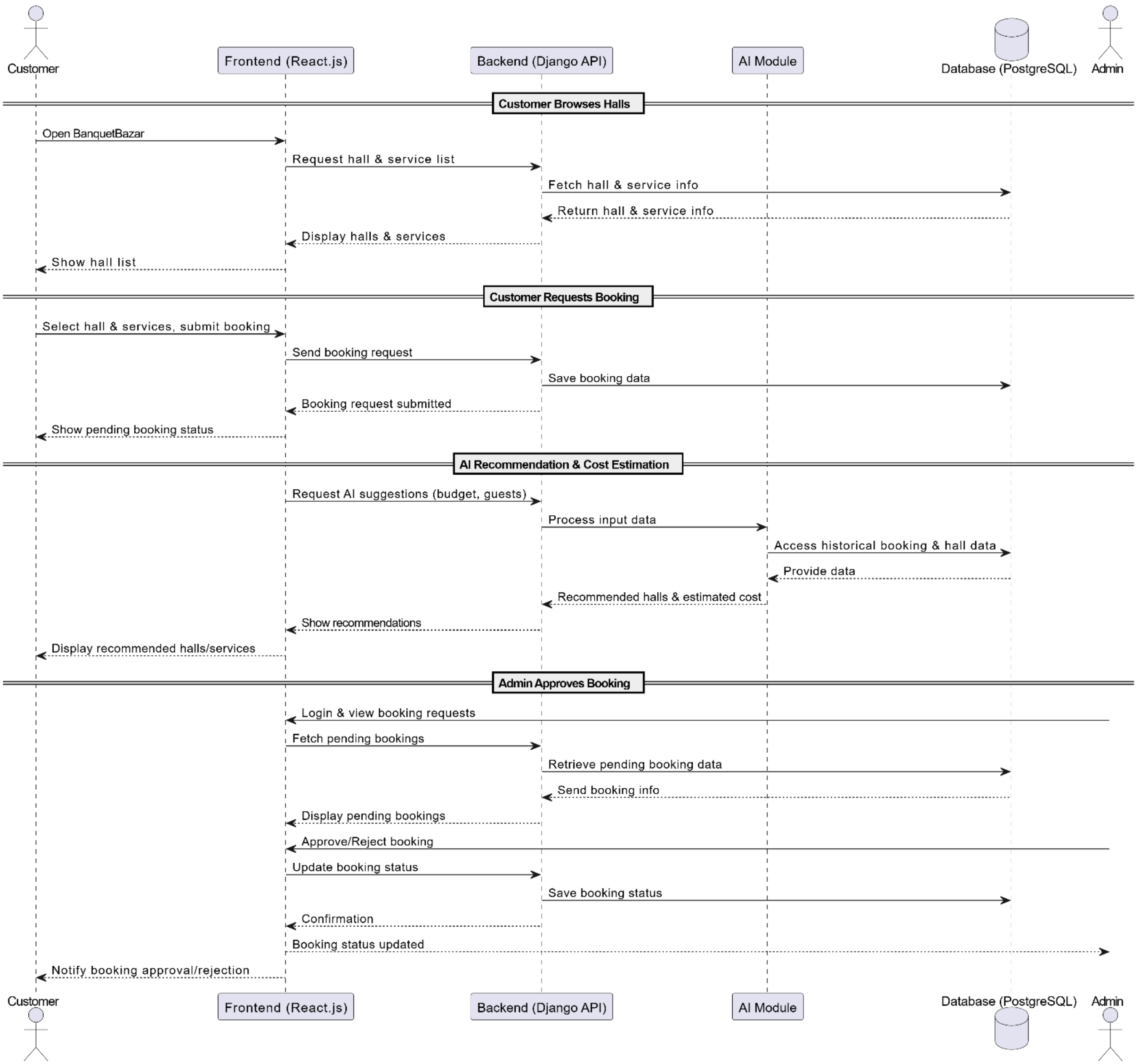

Figure 8: Sequential Diagram

4.6 UI/UX Design

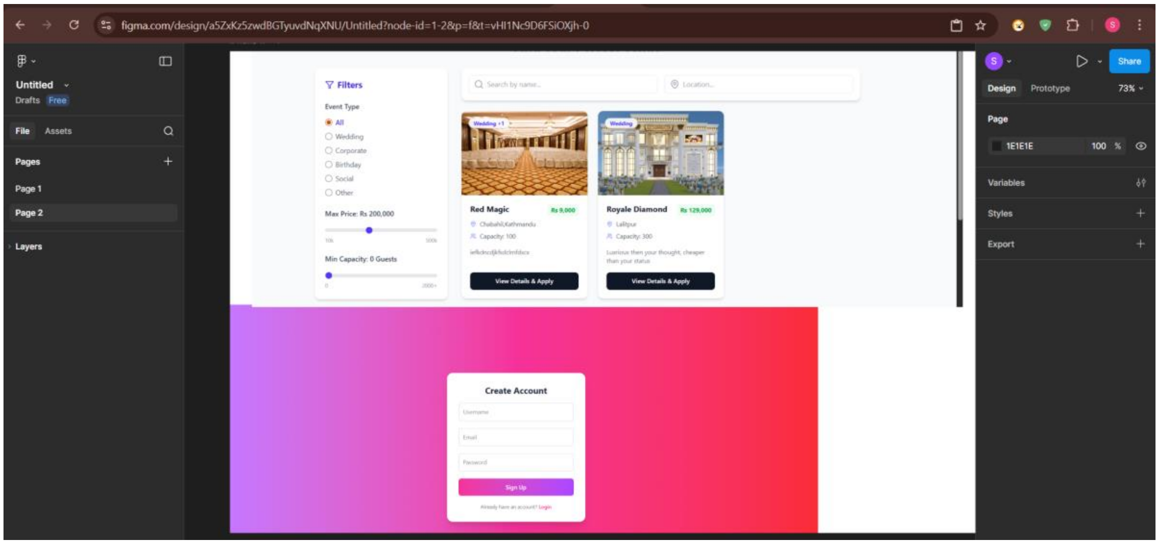

Figure 9: UI/UX Design

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Module Description

The BanquetBazar system is divided into several functional modules to ensure a structured and efficient implementation. Each module is responsible for performing specific tasks within the system, allowing smooth interaction between the frontend, backend, and database.

1) Authentication Module

   This module handles user registration and login functionality for both customers and admins. It ensures secure access to the system using authentication mechanisms and role-based access control. Customers can create accounts and log in, while admins can securely access the admin panel.

2) Hall Management Module

   This module allows admins to manage banquet hall information. Admins can add, update, and delete hall details such as name, location, price, capacity, images, and available services. This ensures that customers always view up-to-date information.

3) Service Management Module

   This module enables admins to manage additional event services such as catering, decoration, photography, and DJ. Admins can add or update service details and pricing, which customers can select current booking.

4) Hall Browsing Module

   This module allows customers to view a list of available banquet halls along with detailed information, including price, capacity, images, and services. It provides an easy-to-use interface for comparing different halls.

5) Booking Module

   This module allows customers to check hall availability, select a hall and services, and submit booking requests. It also enables customers to track the status of their bookings

(Pending, Approved, or Rejected).

6) Booking Management Module (Admin)

   This module allows admins to view all booking requests and manage them efficiently. Admins can approve or reject bookings and provide remarks. It ensures proper coordination between customers and hall owners.

7) AI Recommendation Module

   This module uses a Random Forest algorithm to recommend suitable banquet halls based on user inputs such as budget, guest count, and event type. It helps users make better decisions.

8) Cost Estimation Module

   This module provides automatic cost estimation using a rule-based approach. It calculates the total event cost based on hall price, number of guests, and selected services.

9) Customer Dashboard Module

   This module provides customers with a personalized dashboard where they can view their booking history, current booking status, and AI-based recommendations.

10) Backend API Module

    This module is developed using Django REST Framework and handles communication between the frontend and backend. It manages data processing, authentication, and database operations.

5.2 Frontend Implementation

The frontend of the BanquetBazar system is developed using React.js, a modern JavaScript library for building dynamic and responsive user interfaces. The frontend is responsible for interacting with users, displaying data, and communicating with the backend through APIs. The user interface is designed to be simple, user-friendly, and responsive, ensuring a smooth experience across different devices and screen sizes.

1) Technology Stack

The frontend is built using the following technologies:

- ✓ React.js for creating reusable UI components
- ✓ React Router for navigation between different pages
- ✓ Axios for making HTTP requests to the backend APIs
- ✓ Tailwind CSS for styling and responsive design

2) Component-Based Architecture

The frontend follows a component-based architecture, where the UI is divided into reusable components such as Navbar, Footer, Hall Cards, Booking Forms, and Dashboard panels. This improves code reusability, maintainability, and scalability.

3) User Interface Pages

The main pages implemented in the frontend include:

- ✓ Home Page: Displays featured halls and general information about the platform
- ✓ Login/Signup Page: Allows users to register and log in securely
- ✓ Hall Listing Page: Shows all available banquet halls with details like price, capacity, and services
- ✓ Hall Detail Page: Provides detailed information about a selected hall
- ✓ Booking Page: Allows users to select services and request bookings
- ✓ Customer Dashboard: Displays booking history, status, and AI recommendations
- ✓ Admin Dashboard: Enables admins to manage halls, services, and bookings

4) API Integration

The frontend communicates with the backend using RESTful APIs developed in Django. Axios is used to send HTTP requests (GET, POST, PUT, DELETE) for fetching and updating data such as halls, bookings, and user information.

5) State Management

React's built-in state management (useState and useEffect hooks) is used to manage application data, including user authentication status, booking details, and fetched API data. This ensures dynamic updates of the UI without page reloads.

6) Form Handling and Validation

Forms such as login, signup, and booking requests are implemented with proper validation to ensure correct user input. This improves data accuracy and user experience.

7) Responsive Design

The frontend is designed to be responsive using Tailwind CSS, allowing users to access the system on desktops, tablets, and mobile browsers without usability issues.

5.3 Backend Implementation

The backend of the BanquetBazar system is developed using Django, a powerful Python web framework, along with Django REST Framework (DRF) to build RESTful APIs. The backend is responsible for handling business logic, managing data, processing user requests, and ensuring secure communication between the frontend and the database.

1) Technology Stack

The backend is built using:

- ✓ Django (Python): For handling server-side logic and application structure
- ✓ Django REST Framework (DRF): For creating RESTful APIs
- ✓ PostgreSQL: For database management
- ✓ JWT : For secure user authentication

2) API Development

RESTful APIs are developed using Django REST Framework to allow communication between the frontend and backend. These APIs handle operations such as:

- ✓ User registration and login
- ✓ Fetching banquet hall data
- ✓ Managing booking requests
- ✓ Handling services and admin operations
- ✓ HTTP methods such as GET, POST, PUT, and DELETE are used to perform CRUD (Create, Read, Update, Delete) operations.

3) Database Interaction

Django's ORM (Object Relational Mapping) is used to interact with the PostgreSQL database. Models are created for entities such as Users, Halls, Services, and Bookings. The ORM simplifies database queries and ensures data integrity.

4) Authentication and Authorization

The backend implements secure authentication for both customers and admins. Role-based access control is used to restrict certain functionalities (such as hall management and booking approval) to admin users only. Passwords are securely stored using hashing mechanisms.

5) Business Logic Implementation

The backend processes all core functionalities of the system, including:

- ✓ Checking hall availability for selected dates
- ✓ Managing booking requests and status updates
- ✓ Assigning services to bookings
- ✓ Handling admin actions such as approval or rejection of bookings

6) AI Model Integration

The backend integrates the AI module developed using Python. The trained Random Forest model is loaded using tools like joblib or pickle and used to generate hall recommendations based on user inputs. A rule-based system is used for cost estimation.

7) Error Handling and Validation

Proper validation is implemented for all incoming data to ensure accuracy and prevent errors. The system handles exceptions gracefully and returns meaningful error messages to the frontend.

8) Security Measures

The backend includes security features such as:

- ✓ Password hashing
- ✓ Input validation
- ✓ Protection against unauthorized access
- ✓ Secure API endpoints

5.4 Database Implementation

The BanquetBazar system uses PostgreSQL as the relational database to store and manage all application data efficiently. The database is designed to ensure data integrity, consistency, and fast retrieval of information required for system operations.

1) Database Design Approach

The database is designed based on the Entity-Relationship (ER) model, which identifies key entities such as Users, Halls, Services, and Bookings along with their relationships. The design follows normalization principles to reduce redundancy and improve data consistency.

2) Main Tables / Entities

The system consists of the following main tables:

i. User Table:

Stores user information such as user ID, name, email, password, and role (customer/admin).

ii. Hall Table:

Contains details of banquet halls including hall ID, name, location, price, capacity, images, and description.

iii. Service Table:

Stores information about additional services such as catering, decoration, photography, and DJ, along with their prices.

iv. Booking Table:

Records booking details including booking ID, user ID, hall ID, booking date, event date, selected services, and booking status (Pending, Approved, Rejected).

v. Booking Service Table (Junction Table):

Handles the many-to-many relationship between bookings and services, allowing multiple services to be selected for a single booking.

3) Relationships Between Tables

- ✓ One User can make multiple bookings (One-to-Many).
- ✓ One Hall can have multiple bookings (One-to-Many).
- ✓ One Booking can include multiple services, and one service can be used in multiple bookings (Many-to-Many).

4) Database Operations

The database supports all CRUD operations:

- ✓ Create: Adding new users, halls, services, and bookings
- ✓ Read: Retrieving hall details, booking information, and user data
- ✓ Update: Modifying hall details, booking status, and user profiles
- ✓ Delete: Removing halls, services, or bookings when necessary

5) Integration with Backend

Django ORM is used to interact with the PostgreSQL database. Models are defined in Django to represent each table, and queries are handled efficiently without writing complex SQL manually.

6) Data Security and Integrity

- ✓ Passwords are stored in encrypted (hashed) form
- ✓ Constraints such as primary keys and foreign keys ensure data consistency
- ✓ Validation rules prevent invalid or incomplete data entry

5.5 Algorithm Used

The BanquetBazar system integrates basic Artificial Intelligence techniques to enhance decision-making and improve user experience. Two main approaches are used in this system: a Random Forest algorithm for hall recommendations and a rule-based algorithm for cost estimation.

1) Random Forest Algorithm (Hall Recommendation)

The Random Forest algorithm is a supervised machine learning technique used for classification and prediction. It works by combining multiple decision trees to produce more accurate and reliable results.

In BanquetBazar, the Random Forest model is used to recommend suitable banquet halls based on user inputs such as:

- ✓ Budget
- ✓ Guest count
- ✓ Event type

Working Process: A dataset containing hall details (price, capacity, services, ratings, etc.) is used to train the model. The Random Forest algorithm builds multiple decision trees using this data. When a user enters their preferences, the model processes the input and predicts the most suitable halls. The system then displays recommended halls to the user.

Advantages:

- ✓ High accuracy compared to a single decision tree
- ✓ Handles large datasets efficiently
- ✓ Reduces overfitting

2) Rule-Based Algorithm (Cost Estimation)

A rule-based approach is used to calculate the estimated cost of an event. This method applies predefined formulas and conditions to generate results.

Working Process:

The total cost is calculated based on:

- ✓ Price per plate × Number of guests
- ✓ Cost of selected services (decoration, catering, DJ, photography, etc.)
- ✓ Each service has a fixed price stored in the database.
- ✓ The system adds all costs to generate the final estimated amount.

Example Formula:

Total Cost = (Price per plate × Guest count) + Service Costs

Advantages:

- ✓ Simple and fast calculation
- ✓ Easy to implement and maintain
- ✓ Provides instant results to users

3) Tools Used for Implementation

- ✓ scikit-learn: For implementing the Random Forest model
- ✓ Pandas & NumPy: For data preprocessing and handling datasets
- ✓ joblib / pickle: For saving and loading trained models

5.6 Screenshots of System Features

CHAPTER 6: TESTING AND RESULT ANALYSIS

6.1 Test Plan

The test plan defines the strategy and approach used to evaluate the functionality, performance, and reliability of the BanquetBazar system. It ensures that all system components work correctly and meet the specified requirements before final deployment.

1) Objectives of Testing

The main objectives of testing are:

- ✓ To verify that all system modules function correctly
- ✓ To identify and fix bugs or errors
- ✓ To ensure the system meets functional and non-functional requirements
- ✓ To validate the accuracy of AI recommendations and cost estimation

2) Scope of Testing

The testing process covers the following modules:

- ✓ User Authentication (Login/Signup)
- ✓ Hall Browsing and Viewing Details
- ✓ Booking Module (Request and Status Tracking)
- ✓ Admin Panel (Hall, Service, and Booking Management)
- ✓ AI Recommendation Module
- ✓ Cost Estimation Module

3) Types of Testing Performed

The following testing methods are applied:

i. Unit Testing:

Individual components such as forms, APIs, and functions are tested independently.

ii. Integration Testing:

Interaction between frontend, backend, and database is tested to ensure proper data flow.

iii. System Testing:

The complete system is tested as a whole to verify overall functionality.

iv. User Acceptance Testing (UAT):

The system is tested from the user's perspective to ensure it meets user expectations.

v. Performance Testing:

The system's response time and ability to handle multiple users are evaluated.

4) Test Environment

- ✓ Operating System: Windows 10/11
- ✓ Browser: Google Chrome (latest version)
- ✓ Frontend: React.js
- ✓ Backend: Django with Django REST Framework
- ✓ Database: PostgreSQL

5) Test Strategy

- ✓ Both manual testing and basic automated testing are used
- ✓ Test cases are created for all major functionalities
- ✓ Errors are logged and fixed during development
- ✓ Retesting is performed after fixing bugs to ensure system stability

6) Success Criteria

The system is considered successful if:

- ✓ All functionalities work as expected without critical errors
- ✓ The system responds within acceptable time limits (3:5 seconds)
- ✓ AI recommendations and cost estimations produce reasonable results
- ✓ Users can complete booking tasks without confusion

6.2 Test Cases (Table Format)

Test Case ID	Module	Test Scenario	Input	Expected Output	Actual Output	Status
TC01	Authentication	User Registration	Valid user details	User account created successfully	As Expected	Pass
TC02	Authentication	User Login	Correct email & password	User logged in successfully	As Expected	Pass
TC03	Authentication	Invalid Login	Wrong password	Error message displayed	As Expected	Pass
TC04	Authentication	Empty Login Fields	No input	Validation error shown	As Expected	Pass
TC05	Authentication	Duplicate Registration	Existing email	Error message displayed	As Expected	Pass
TC06	Hall Browsing	View Hall List	Click “Browse Halls”	List of halls displayed	As Expected	Pass
TC07	Hall Details	View Hall Details	Select a hall	Detailed hall info shown	As Expected	Pass
TC08	Hall Browsing	No Halls Available	Empty database	Message “No halls available”	As Expected	Pass
TC09	Booking	Check Availability	Select valid date	Availability status shown	As Expected	Pass
TC10	Booking	Check Invalid Date	Past date	Error message displayed	As Expected	Pass

TC11	Booking	Request Booking	Valid booking details	Booking request submitted	As Expected	Pass
TC12	Booking	Invalid Booking	Missing fields	Validation error message	As Expected	Pass
TC13	Booking	Cancel Booking	Cancel request	Booking cancelled successfully	As Expected	Pass
TC14	Admin Panel	Add Hall	Valid hall data	Hall added successfully	As Expected	Pass
TC15	Admin Panel	Add Hall Invalid	Missing fields	Error message shown	As Expected	Pass
TC16	Admin Panel	Update Hall	Modify hall details	Hall updated successfully	As Expected	Pass
TC17	Admin Panel	Delete Hall	Select hall	Hall removed from system	As Expected	Pass
TC18	Admin Panel	View Bookings	Open booking s list	All bookings displayed	As Expected	Pass
TC19	Admin Panel	Approve Booking	Select booking	Status updated to Approved	As Expected	Pass
TC20	Admin Panel	Reject Booking	Select booking	Status updated to Rejected	As Expected	Pass
TC21	Services	Add Service	Valid service data	Service added successfully	As Expected	Pass
TC22	Services	Update Service	Modify service details	Service updated successfully	As Expected	Pass

TC23	Services	Delete Service	Select service	Service removed	As Expected	Pass
TC24	AI Recommendation	Get Suggestions	Budget, guests, type	Suitable halls recommended	As Expected	Pass
TC25	AI Recommendation	Invalid Input	Empty input	Error/No recommendation	As Expected	Pass
TC26	Cost Estimation	Calculate Cost	Guests + services	Total cost calculated correctly	As Expected	Pass
TC27	Dashboard	View Booking Status	Logged-in user	Booking history displayed	As Expected	Pass
TC28	Performance	Load System	Multiple users	System responds within 3:5 sec	As Expected	Pass

6.3 Unit Testing

Unit testing is performed to ensure that each module or component of the BanquetBazar system works correctly independently. Each unit is tested for functionality, input validation, and expected output. The testing was carried out using manual test cases for the frontend, backend, and AI modules.

6.4 Integration Testing

Integration testing is performed to verify that multiple modules of the BanquetBazar system work together correctly. After unit testing confirmed the functionality of individual components, integration testing ensures seamless communication between frontend, backend, AI module, and database.

Modules Integrated	Description	Test Method	Result
Frontend & Backend	Ensuring UI actions trigger correct API calls	Perform hall browsing, booking requests, login/logout, and profile updates	Passed : All frontend actions successfully interact with backend APIs
Backend & Database	Check data storage, retrieval, and updates	Add new halls, services, and bookings	Passed : Data correctly inserted, updated, and deleted without integrity issues
Booking Module & AI Recommendation	AI provides recommendations during booking	Input event type, budget, guest count	Passed : Recommendations displayed correctly and cost estimation calculated accurately

Admin Panel & Database	Admin CRUD operations affect database	Add/update/delete halls and services	Passed : Changes correctly reflected in database; validation applied
Customer Dashboard & Backend	Display booking status and AI suggestions	Login as customer and check dashboard	Passed : Past and upcoming bookings shown correctly; AI suggestions visible
Service Selection & Cost Estimation	Selecting services updates total cost	Select multiple services and check cost	Passed : Total cost updates dynamically and matches AI cost estimation
API Integration	Frontend consuming backend REST APIs	Test GET, POST, PUT, DELETE endpoints	Passed : All endpoints responded with correct data and status codes
Error Handling Across Modules	Test invalid inputs, unavailable halls, or missing data	Input wrong dates, empty fields, or unavailable halls	Passed : Error messages shown correctly, system prevented invalid operations
Authentication & Role Management	Ensure users and admins have correct permissions	Login as customer/admin and attempt restricted actions	Passed : Role-based access enforced, unauthorized actions blocked
End-to-End Booking Flow	From hall browsing → service selection → booking request → admin approval	Complete booking as customer, approve as admin	Passed : End-to-end booking successfully completed with correct notifications and database updates

6.5 System Testing

System testing is conducted to validate the complete BanquetBazar system as a whole, ensuring that all functional and non-functional requirements are met. It focuses on evaluating the system's behavior, reliability, performance, and overall user experience in real-world scenarios.

Test Type	Description	Test Scenarios	Expected Result	Actual Result
Functional Testing	Verify that the system performs all required functions	Customer login/signup, hall browsing, booking requests, AI recommendations, admin panel operations	All functions work according to requirements	Passed
Usability Testing	Check the user interface for ease of use	Navigate through hall listings, booking process, dashboard, admin panel	Users can perform tasks intuitively without confusion	Passed
Compatibility Testing	Ensure system works on different browsers and devices	Test on Chrome, Firefox, Edge, Safari; desktop and mobile browsers	System displays correctly and functions on all tested browsers	Passed
Security Testing	Validate system security	Test role-based access, login with invalid credentials, data encryption	Unauthorized access blocked, passwords encrypted	Passed
Performance Testing	Evaluate system response and load handling	Browse halls, submit bookings with multiple users concurrently	System responds within 2:3 seconds, no crashes	Passed

Stress Testing	Test system under extreme conditions	Simulate high number of users accessing simultaneously	System remains stable; minor delays acceptable	Passed
Error Handling Testing	Check system response to invalid inputs	Submit forms with missing fields, incorrect data, or unavailable dates	System displays appropriate error messages and prevents invalid actions	Passed
Integration Verification	Confirm smooth operation between modules	Frontend-Backend-AI-Database interactions	Data flows correctly, AI recommendations match inputs	Passed
Regression Testing	Ensure new features do not break existing functionality	After adding AI recommendations and cost estimation	All previously tested features continue to function correctly	Passed
End-to-End Testing	Validate complete user journey	Customer searches halls → selects services → requests booking → admin approves → final confirmation	Booking completed successfully with correct cost and recommendations	Passed

6.6 Performance Evaluation

Performance evaluation measures how efficiently BanquetBazar operates under different conditions. It ensures the system can handle multiple users, large datasets, and AI computations without significant delays, providing a smooth experience for both customers and admins.

Evaluation Criteria

1) Response Time: Time taken for the system to respond to user actions such as login, hall browsing, booking requests, and AI recommendations.
2) System Throughput: Number of simultaneous requests handled by the system without performance degradation.
3) Database Query Performance: Efficiency in retrieving hall, service, and booking data from PostgreSQL.
4) AI Recommendation Accuracy and Speed: Time taken to provide hall suggestions based on budget, guest count, and event type.
5) Scalability: System's ability to handle increasing number of halls, bookings, and users.

Observations:

- ✓ The system maintains high performance with fast response times across all core functionalities.
- ✓ AI recommendations and cost calculations are efficient and provide results almost instantly.
- ✓ The database handles multiple concurrent queries without noticeable delay.
- ✓ System remains stable under high user load, proving reliability for small to medium-scale deployments.
- ✓ Minor latency is observed when scaling up halls and bookings, which can be optimized in future updates.

CHAPTER 7:CONCLUSION AND RECOMMENDATION

7.1 Summary

BanquetBazar is a comprehensive web-based AI-powered banquet management system designed to modernize and simplify the traditional banquet hall booking process. The system centralizes information on halls, services, and availability, allowing customers to easily browse options, request bookings, and receive intelligent recommendations based on their budget, guest count, and event type.

The project follows an Incremental Development Model, ensuring that each module—from user authentication to AI recommendations—was developed, tested, and integrated in stages. The frontend was built with React.js, providing a responsive and user-friendly interface, while the backend was developed using Django and Django REST Framework, ensuring secure and efficient data handling. A PostgreSQL database stores all system data, including users, halls, services, bookings, and AI training datasets.

AI features, including Random Forest-based hall recommendations and rule-based cost estimation, were implemented to enhance decision-making and improve event planning efficiency. The system was evaluated through unit testing, integration testing, system testing, and performance evaluation, demonstrating fast response times, reliability, and scalability for small to medium-sized user loads.

In conclusion, BanquetBazar successfully addresses the challenges of manual banquet hall booking by providing a centralized, intelligent, and user-friendly platform for customers and administrators. It bridges gaps in existing systems, such as fragmented processes, lack of intelligent recommendations, and absence of automatic cost estimation, while providing a foundation for future enhancements like mobile apps, payment integration, and advanced analytics.

7.2 Conclusion

The BanquetBazar project demonstrates the successful development of a web-based AI-powered banquet management system that addresses the inefficiencies and limitations of traditional banquet hall booking methods. By integrating modern web technologies with artificial intelligence, the system provides a centralized platform where customers can browse halls, check availability, select services, and receive intelligent recommendations.

Through the implementation of React.js on the frontend and Django with Django REST Framework on the backend, supported by a PostgreSQL database, the system ensures a seamless, secure, and responsive user experience. The AI module, which leverages Random Forest algorithms for hall recommendations and rule-based cost estimation, enhances decision-making and simplifies event planning for users.

Testing and performance evaluations indicate that the system is reliable, efficient, and user-friendly, capable of handling typical user workloads without significant delays or errors. Overall, BanquetBazar successfully fulfills its objectives of improving transparency, convenience, and efficiency in banquet hall bookings. The project also provides a strong foundation for future enhancements, including mobile application development, online payment integration, and advanced analytics for administrators.

7.3 Limitations

Despite its successful implementation, the BanquetBazar system has certain limitations in its current version:

i. Web-Only Platform: The system is accessible only via web browsers; no dedicated mobile application is available for smartphones or tablets.

ii. Payment Gateway Integration: Online payment functionality is not implemented, requiring users to complete transactions offline.

iii. Real-Time Communication: The platform does not support live chat or direct messaging between customers and hall owners.

iv. Limited Admin Features: Advanced features such as multi-branch management, detailed analytics, and notifications are not included.

v. AI Model Limitations: Recommendations and cost estimations rely on historical data and simple models; results may vary with incomplete or inconsistent data.

vi. Scalability Constraints: The system is designed for a moderate number of halls and bookings; extremely high traffic may require further optimization.

These limitations provide areas for improvement and form the basis for future enhancements of the system.

7.4 Future Enhancements

To further improve the BanquetBazar system and provide a more comprehensive experience, the following enhancements can be implemented in the future:

i. Mobile Application Development: Create a dedicated mobile app for Android and iOS to allow users to access the platform on-the-go.

ii. Payment Gateway Integration: Enable secure online payments through popular payment gateways to streamline the booking process.

iii. Real-Time Communication: Introduce live chat or messaging between customers and hall owners for instant support and clarification.

iv. Advanced AI Models: Implement more sophisticated AI algorithms, such as collaborative filtering or deep learning, for improved hall recommendations and dynamic cost estimation.

v. Multi-Branch Management: Allow banquet owners to manage multiple halls across different locations from a single admin account.

vi. Notification System: Integrate email, SMS, or WhatsApp notifications for booking confirmations, reminders, and updates.

vii. Analytics and Reporting: Provide detailed analytics and performance reports for admins to track bookings, revenue, and customer preferences.

viii. Integration with External Services: Connect with catering, decoration, or event management service APIs to offer a complete event planning solution.

ix. Enhanced User Experience: Continuously improve UI/UX based on user feedback, including personalized dashboards and interactive hall visualizations.

These future enhancements will ensure BanquetBazar remains scalable, user-friendly, and competitive in the evolving event management market.

References:

1. Orazova, A., & Permanova, D. (2025). A digital banquet hall management system for event planning and resource optimization. International Journal of Information Systems and Applications, 12(1), 45:56. https://mpcareer-google.ru/index.php/journal/article/view/619

2. Yalini, A. V. E. (2017). Web-based wedding hall management system for *Loyolas* Wedding Hall, Jaffna (Undergraduate thesis). University of Colombo School of Computing. https://dl.ucsc.cmb.ac.lk/jspui/handle/123456789/3993

3. Lalitha, R., Kumar, S., & Priya, M. (2022). Design and implementation of an online event hall booking and reservation system. Journal of Web Engineering and Technology, 8(3), 112:120. https://ieeexplore.ieee.org/abstract/document/10040388

4. Islam, F. (2024). AI driven development of modern web application (Bachelor's thesis, Tampere University of Applied Sciences). Theseus. https://urn.fi/URN:NBN:fi:amk-2024060922410

5. React Documentation. (2025). React : A JavaScript library for building user interfaces. Facebook. https://reactjs.org/docs/getting-started.html

6. Django Software Foundation. (n.d.). Django documentation. https://docs.djangoproject.com/en/stable/